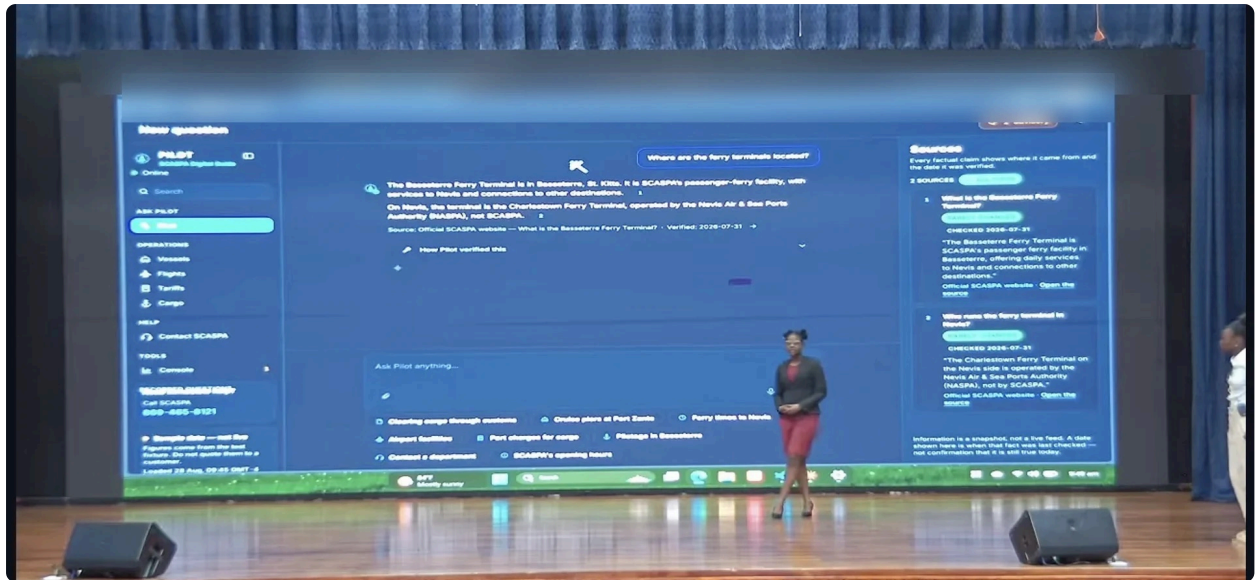




EASTERN CARIBBEAN CENTRAL BANK (ECCB)

Islands of Intelligence

ECCU Generative AI & Python Summer Camp (Buildathon) 2026

How young Caribbean builders shipped working AI against real organisational challenges.

WRITTEN BY

Ms T. Valerie Onu

Instructional Designer, Lead Trainer and SKN Hub Co-Coordinator

Mrs Golda Palmer

Instructional Design and SKN Hub Co-Coordinator

8

territories

26

Agile Pods

11

client organisations

20

teaching days

Regional showcase · 28 August 2026 · ECCB HQ, St Kitts · streamed on ECCB Connects



Read the online edition

Scan to read the interactive edition online.

eccuaicamp2026.netlify.app/islands/en/

Contents

FRONT MATTER

— Opening Statement

They did not come to build school projects.

PART I · THE JOURNEY

01 The Camp That Became a Consultancy

The year notional clients became real enterprise ones, and 50+ campers earned freeCodeCamp Python Certification.

03 Slaying the Dragon

Six rotating hats, and the reform that stopped one camper doing all the coding.

04 Knowledge vs Memory

Every hand goes up when you ask if the distinction is understood. A Kahoot on day six said otherwise.

06 The Golden Record

Topic · Question · Hard Fact · Source URL · Last Verified.

07 From Chatbot to Crew

Six components, then guardrails, then grounding, then a crew of agents.

09 Ship It Live

A live deploy failed on a wrong start command, and was fixed on stage.

PART II · THE BUILDS

02 The Bot that delivers Truth You Could Read Line by Line

AquaCoders put their Python backend on the showcase screen rather than a polished slide.

05 Boy, How Much Taxes Me Got Pay Now?

A.I.D.A. answered the question everyone was already asking, in the dialect they asked it in.

08 A Bot That Changes Who It Is

Age-tiered guides, because a six-year-old and a seventeen-year-old need different answers.

10 The Port That Never Sleeps

PILOT answers in about a second, and knows exactly when to pass you to a human.

11 The Chatbot Showcase Field

All eleven chatbot presentations.

PART III · THE ARGUMENT

12 **Honest Edges**

Eight edges, owned in public — and why they are the programme, not its failures.

13 **The Showcase**

Panel, partnership, results.

14 **Why This Works**

Authentic youth engagement and the innovation ecosystem.

15 **What Comes Next**

The alumni pathway.

16 **The Macro Impact: Digital Sovereignty and the Forward Deployed Engineer**

Why the Eastern Caribbean's economic future depends on the blueprint built at this camp.

Opening Statement

They did not come to build school projects.

They did not come to build school projects. They came to solve real problems.

This was not a conventional summer camp. It was a regional buildathon-style bootcamp: intensive, team-based, client-facing and product-oriented. Campers learned by building, debugging, testing and shipping working AI systems against real organisational challenges — four weeks of intensive learning and building, then two weeks to refine, rehearse and prepare for the regional showcase.

In the summer of 2026, young people from eight Eastern Caribbean territories were handed something most adults never get: a real client, a real problem, and the mandate to ship something that worked — a working AI assistant for a bank, a port, a water company, a tax department. Organisations their own families depend on.

Eleven clients. Twenty-six Agile Pods. One regional showcase, tested before a panel of technology, climate and youth-development professionals and watched by visiting ambassadors.

This is the story of what happened when Caribbean teenagers stopped being taught *about* technology and were trusted *to build* it — and, just as importantly, were allowed to fail in public and fix it in front of everyone.

The Camp That Became a Consultancy

The year notional clients became real enterprise ones, and 50+ campers earned freeCodeCamp Python Certification.

Something changed in 2026, and it was not the code. Campers still learned Python. Two other things changed, and both of them raised the stakes.

The first was who they were building for.

Earlier years ran on notional clients: a plausible organisation, invented for the exercise, which could not be disappointed because it did not exist. 2026 replaced them with real ones — and not small ones. Eleven **enterprise clients**: banks, a ports authority, a water authority, revenue departments, a national insurance scheme. Organisations with real customers, real obligations and real reputations, which published a brief, received a build, and then sat in judgement on it.

That substitution changes everything downstream. A notional client cannot tell you that your answer would embarrass them. A real bank can, and did.

The second was the credential they left with.

Previous cohorts finished with a Python certificate from a credible online institution or platform. This year campers sat the official freeCodeCamp Python examination, and 50+ campers gained Python Certification — a globally recognised credential, publicly verifiable, with a certificate URL for each camper who passed.

The difference is in what the credential asks of you. freeCodeCamp's Python certification is earned by working through a substantial block of projects — roughly three hundred hours — and the certification projects in particular require applying the whole curriculum rather than following along with it. It costs the camper nothing. And it ends in a public, shareable verification link.

That combination is rare: a credential that is free to earn, hard to earn, and checkable by anyone — an employer, a university, a client — without having to take the camp's word for it, or the camper's.

It is also, quietly, the best preparation the camp could have given them for what came next. A project-first credential and a real enterprise brief ask for the same thing: not whether you followed the lesson, but whether you can build the thing.

Teams were structured like small technology companies. Each had a client. Each had a brief published on a board anyone could read. And the curriculum was reverse-engineered: rather than marching forward through Python syntax toward some eventual project, the facilitators started from a finished product and worked backwards to the Python it required. You learned a loop because your bot needed one, not because loops came next.

The shift began in a planning session, with a question that sounded like heresy. *Can I be the devil's advocate for a moment?* asked Telbert Pyke, a facilitator on the camp and its IT support. *Have we thought about feeding all of this information into one of these known AI.*

It is worth sitting with what that question implies. If a general-purpose assistant can already answer a customer's question, what exactly is a camp full of teenagers building? The answer the programme arrived at — and then spent twenty days proving — is that a general model will answer anything, including things it has invented, and that an organisation cannot put its name on that. What the teams built was not intelligence. It was accountability: bots that knew where their facts came from, refused questions outside their remit, and handed you to a human when the stakes required one.

Why enterprises build it this way

That is not a teaching simplification. It is the reason retrieval and agentic architectures exist in industry at all.

A bank cannot deploy a system that improvises a balance. A revenue department cannot deploy one that improvises a tax rule. The moment an institution is accountable for what its software says, the consumer chatbot pattern — one model, answering from memory, in one step — stops being deployable. Two patterns replace it, and both were in this curriculum.

Retrieval-augmented generation turns the model from the source of the answer into the interface to it. The facts live in a corpus the organisation owns and governs — policy circulars, published tariffs, statutes, rate tables — and the system retrieves from that corpus before it speaks, then says where the answer came from. The value is not that it makes the model cleverer. It is that it makes the model *auditable*: someone can ask where a figure came from and get a document, not a shrug. That is why the camp's first rule was that a fact must be sourced before the bot may say it, and why the Golden Record carries a *last verified* column.

Agentic orchestration splits the work into nodes with defined jobs and explicit handoffs — classify the request, retrieve, check compliance, verify, escalate — rather than asking one model to do everything at once. Each step can be constrained, logged and tested on its own, and any step can refuse or hand off. It is the difference between producing an answer and running a process, and a process is the thing an organisation can put controls on.

Both patterns exist for the same reason, and it is not sophistication. It is liability. Retrieval buys provenance; orchestration buys control. Together they are what makes a language model something a regulated institution can actually put its name on.

Which is why the shapes recur across every build in this book. **W.E.S** refusing to return a balance without a valid token. **A.I.D.A.** wrapping a tax estimate in a mandatory disclaimer and routing anything harder to a staff console. **ASPIRE AI** adding output-schema checkers after its agents got tangled in testing. **PILOT** showing its sources with the date each was last checked, and handing anything touching customs to a human dispatcher.

None of those are novice flourishes. They are the working practices of the field, arrived at by teenagers who had a real client and therefore a real reason to need them.

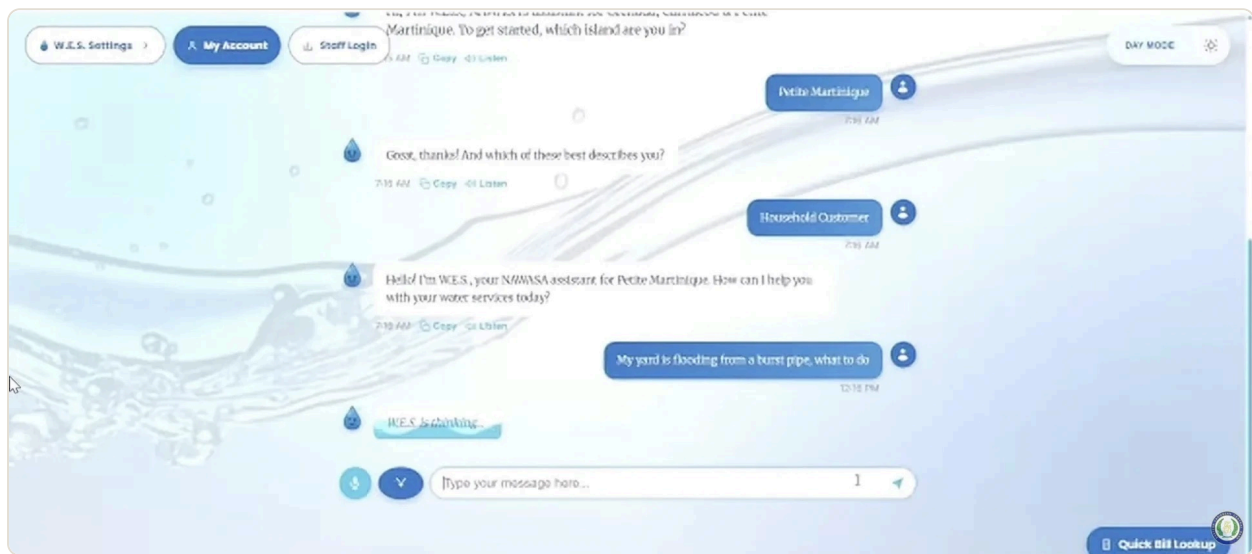
That is a consultancy problem, not a coding exercise. And it is why, by late August, several teenage-led teams could stand in front of a regional technical panel and answer cross-examination about prompt injection.

The Bot that delivers Truth You Could Read Line by Line

FIRST PLACE · AQUACODERS × NAWASA

AquaCoders · NAWASA · SVG

🏆 1st Place Chatbot · 73 points



A household customer in Petite Martinique reports a burst pipe. W.E.S routes the emergency before it asks for a name.

THE PROBLEM

“It's 6:00 a.m., Patrick's yard just flooded, and he has no idea who to call.

AQUACODERS / W.E.S

He does not want a phone tree, an FAQ, or an apology. He wants something that will take the report now and tell him it is being handled — and that exact moment is what W.E.S was built for.

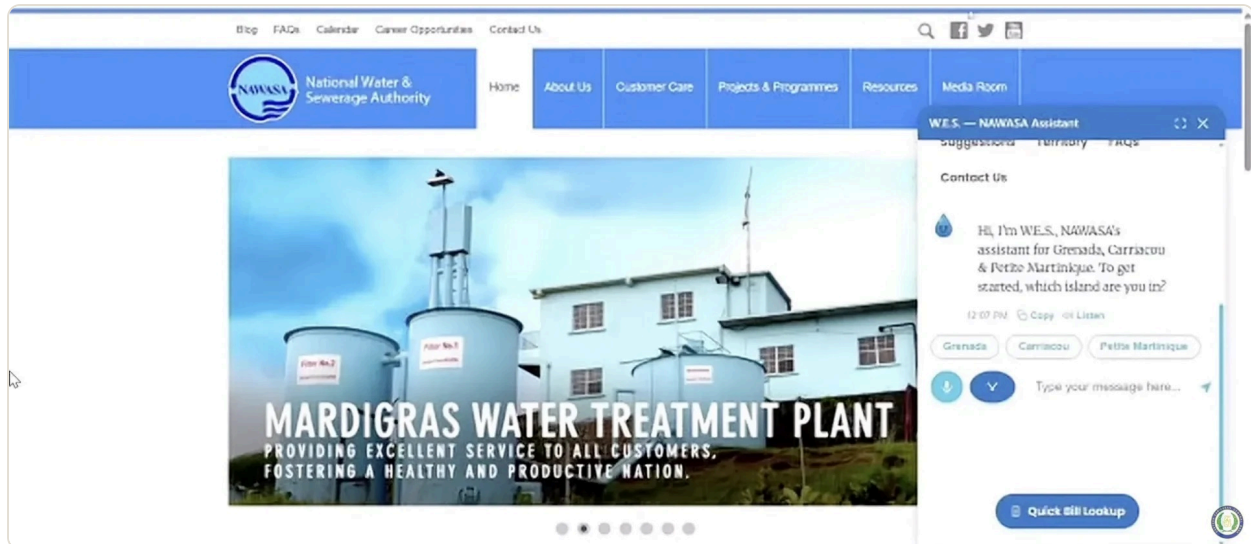
A team from St Vincent and the Grenadines built a water assistant for a water authority in Grenada. They had never walked its pipes, never stood in its offices, never met the customers who would use it. What they had was the brief — *a bot that can read a water meter with its eyes* — and four weeks.

They took the brief literally. Not as a metaphor for convenience, but as the one capability the thing had to actually do.

WHAT THEY BUILT

W.E.S runs inside a live simulation of the website of the National Water and Sewerage Authority (NAWASA), over a 628-fact knowledge base spanning Grenada, Carriacou and Petite Martinique. Six user personas each get an adaptive avatar — a hardhat for contractors, a cane for elderly and vulnerable customers. Emergency routing skips the FAQ entirely and issues a hotline, a WhatsApp dispatch link and a unique reference code before it asks your name. A phone photo of a meter is read by optical character recognition (OCR), and re-taken if blurry rather than guessed. A fifteen-question conversational intake outputs a print-ready PDF connection form. Billing spikes raise proactive leak warnings, and a community outage feed carries advisories the island can react to.

628 verified facts across Grenada, Carriacou and Petite Martinique	6 user personas household, commercial, field worker, elderly and vulnerable
3 islands served Grenada · Carriacou · Petite Martinique	73 points, first place highest score in the Chatbot Division



W.E.S embedded in NAWASA's own site, greeting a visitor and asking which island they are on.

WHAT THE JUDGES PRESSED ON

Horace Lashley of the Caribbean Institute for Meteorology & Hydrology (CIMH) asked two questions.

"When will live integration occur with NAWASA's public servers?" The webhooks and REST endpoints, the team answered, were code-complete — waiting only on NAWASA's new portal and billing application programming interface (API).

"How confident are you in authentication and preventing data leaks?" Retrieval isolates general knowledge from private records, they said, and a token is verified before any handoff occurs.

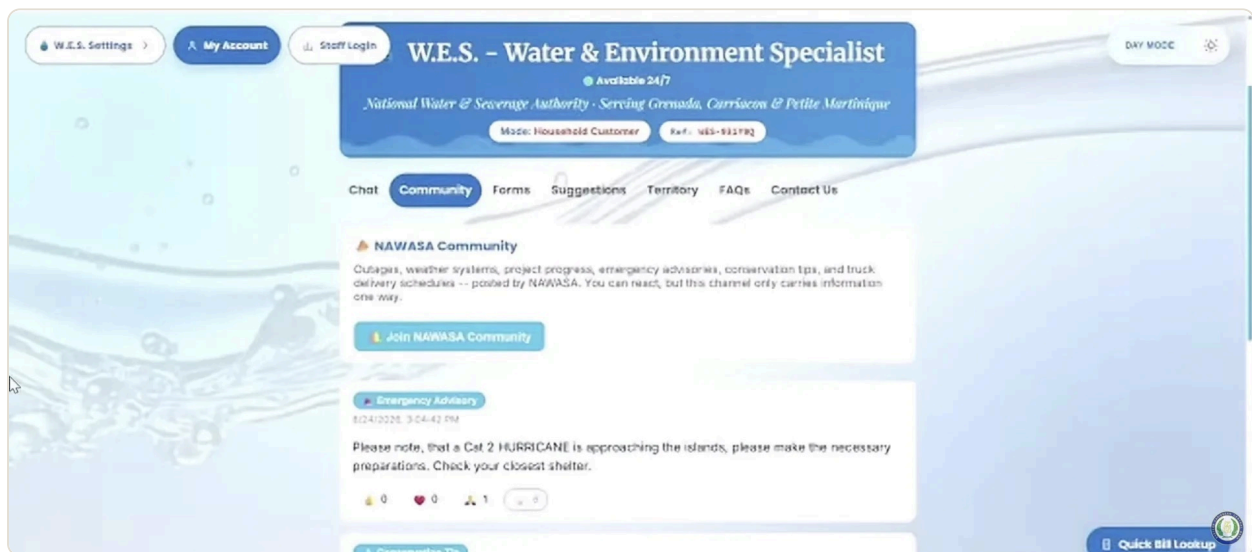
Neither answer was a promise about the future. Both were statements about code that already existed, which the team was willing to show.

THE TURNING POINT

Asked to prove it, the team did not reach for a slide. They put their Python backend on the showcase screen and read it aloud, line by line.

“We'd rather show you the truth than a polished lie ... No token raises an error and refuses to run. It cannot invent a balance. That is what agentic means in practice!”

AQUACODERS / W.E.S



The community feed carrying a Category 2 hurricane advisory — information the island can react to, one way only.

BUILT WITH

Python

FastAPI

LangGraph

RAG

OCR

Gemini

THE TEAM

Alysson Ambrose

Azaray Defreitas

Akasha Dember

Mikayla Durant

J'Nae Edwards

Cornel London

Aren Ollivierre

Arriana Peters

Kyron Quashi

Shannon Ralph — Team Leader

Keleisha Simmons

Kevin Thomas

Nijonte Thomas

Trust became something you could read line by line.

EDITORIAL REVIEW

Our reading of the build

An independent assessment by the editors of this publication — not the judges' scoring, and not the team's own account.

Craft

The build is unusually complete for four weeks. Most projects at this level stop at a working conversation; W.E.S ships a knowledge base, an intake flow that produces a real document, an image pipeline and a public feed. The detail that gives the game away is the OCR behaviour — a blurry meter photo is re-taken rather than guessed. That is a decision about failure, and teams that design for failure are building products rather than demos.

What they learnt, and where it shows

The twenty-day arc is legible in the product almost line for line. The guardrails week appears as the token check that refuses to run rather than invent a balance. The Golden Record appears as 628 facts carrying their own sources. Retrieval appears as the wall between general knowledge and private records. Agent handoffs appear as emergency routing that dispatches before it collects a name.

They did not work through the syllabus as a checklist; they used it to make specific decisions about one specific client.

Usability

The strongest decision is an ordering one. In an emergency W.E.S issues a hotline, a dispatch link and a reference code *before* it asks who you are. Almost all service software asks for identity first. Reversing that is a judgement about what a frightened person at six in the morning actually needs, and it is the clearest evidence in the build that the team thought about a user rather than a feature list. The persona avatars — a hardhat, a cane — risk being decorative, but paired with adaptive language they do real work for people who are not confident online. The one-way community feed is honest about its own limits; it does not pretend to be support.

Scalability

Two things stand between W.E.S and production, and the team named both rather than hiding them. The integration is code-complete but unproven: the webhooks and REST endpoints exist, NAWASA's portal and billing API do not yet, and until they connect the billing features remain a demonstration. Second, 628 facts are a maintenance surface rather than an asset — somebody at NAWASA has to own them, and a "last verified" column is only ever as good as the person updating it. The architecture itself travels well: swap the territory data and the shape holds. The operating cost travels with it.

“Just text WES, he ready to chat — check your bill, just like that!

AQUACODERS / W.E.S



Read the WES chapter

Scan to open this chapter on your phone.

[eccuaicamp2026.netlify.app/islands/en/bo
ok/wes-aquacoders/](https://eccuaicamp2026.netlify.app/islands/en/bo
ok/wes-aquacoders/)

Images captured at the 2026 ECCU Chatbot & Multimedia Showcase, 28 August 2026.
Screen captures have been cropped to remove browser and streaming interface; the
chatbot interfaces and the presenters are unaltered.

Slaying the Dragon

Six rotating hats, and the reform that stopped one camper doing all the coding.

The build needed a shape to hang itself on — something to make four weeks of Python feel like it was going somewhere. Nicole Johnson, the camp's Python lead, proposed one: *a quest to slay something ... let's make it chatbot style ... it's a little fun, it's challenging.* Golda Palmer, who carried the pastoral-care and guardrails brief, answered: *I love it. I love it, I love it, I love it.*

Underneath the framing sat a reform that mattered more.

The previous year had produced a failure mode nobody designed and everybody recognised. One camper would be identified early as the one who codes. The rest would quietly become an audience — present, willing, and not writing a line.

Last year, once people heard who the builder was, they stopped coding, T. Valerie Onu — Telly, the camp's lead coordinator and instructional designer — told the room on day one.

The fix for 2026 was structural rather than motivational. Nobody was asked to try harder, or reminded that everyone's contribution matters. Instead every pod was given **six hats** and required to pass them around: *every week your roles are going to switch. So all of you will be coding nonetheless.*

The design does two things at once. Because the rota is fixed in advance, no camper has to volunteer for the coding seat — it simply arrives, and arrives for everyone. And because several of the hats are not engineering roles at all, a camper who turned up certain they were not a coder still had somewhere to contribute from strength in week one. Path Clearer clears obstacles. Client's Voice holds the brief. Feelings Advocate watches how the team is actually doing. Builder writes the code.

By week two, the rota had them all writing it, because it did not care how they saw themselves.

The dragon was never really the code. It was the belief, arriving on day one and already fully formed, that some people in the room were the technical ones and the rest were there to watch.

Knowledge vs Memory

Every hand goes up when you ask if the distinction is understood. A Kahoot on day six said otherwise.

Ask a room of learners whether they understand the difference between what a chatbot **knows** and what it **remembers**, and every hand goes up. The words are ordinary. The distinction sounds obvious.

Then ask which of the two is doing the work when a customer says *and what about the other one?* — and the hands come down.

That space, between the idea a learner can nod at and the idea they can use, is where most teaching about AI quietly fails. Someone explains a concept, shows a diagram, runs a demonstration that works, and the room agrees. Agreement is cheap. Nothing has been asked of anybody yet, so nothing has been revealed.

The camp was built to stop that gap staying hidden. Its method was immersive in a particular sense — not immersive as in elaborate, but immersive as in *you are inside the consequence*. Roles rotated, so nobody could watch somebody else code. The client was real, and would read the result. The sample code had been broken on purpose. And the assessments were designed to expose what campers had not understood rather than to confirm what they had.

Teaching that way has a cost: it surfaces problems a lecture would have left buried, and then somebody has to deal with them. In week two the camp paid it.

What the Kahoot exposed

There is a distinction the whole programme turns on. **Knowledge** is what a system can look up. **Memory** is what it carries forward through a conversation. Confuse them and you build a bot that either forgets who it is talking to, or invents what it should have retrieved.

A Kahoot revealed that the cohort had not got it. The scores came back and the gap was not a few campers having an off morning — they did not understand memory, and had not understood it for some time.

By day six the verdict was blunt: *We need to take a whole lesson on memory ... the students don't understand it.*

Losing a day on purpose

What happened next is the part that matters. The lesson stopped and the concept was re-taught on the spot, at the cost of the day's plan. In a programme running to a fixed twenty-day schedule with a fixed showcase date, that is an expensive decision. It was made anyway.

The re-teach also changed method. Terbry Lewis, a facilitator from Saint Lucia, seeded the sample code with deliberate errors — *one or two*, introduced on purpose — so that campers could not run it and move on. *They wrestle with it*, he said, *which was nice*.

That is the immersive principle applied to a repair rather than a lesson plan. A camp can explain memory in ten minutes. What it cannot do in ten minutes is give someone the experience of having been wrong about it, which is the thing that actually sticks.

Try it yourself

Reading a definition is not the same as feeling the difference. The campers did not learn this from a sentence — they learned it from getting a question wrong in front of each other, and then from code that would not run until they understood why.

So before you go on: take the question, then switch a bot's memory off and watch a conversation that was working a moment ago fall apart.

Interactive element — open this chapter online to try it.

What it bought

Every grounded answer, every source-checked claim and every context-preserving language switch later in this book rests on a cohort of teenagers having got this distinction straight in the second week.

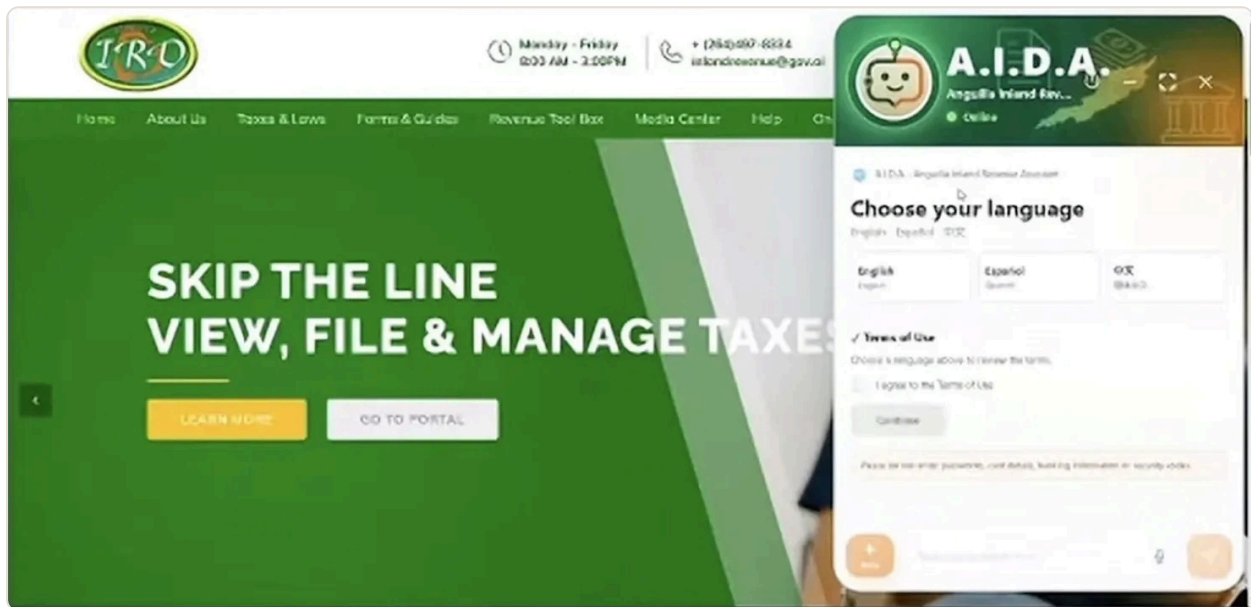
They did — because someone was willing to lose a day.

Boy, How Much Taxes Me Got Pay Now?

SECOND PLACE · ISLAND WAVES × IRD ANGUILLA

Island Waves · Inland Revenue Department, Anguilla · MNI/AXA/Nevis

🏆 2nd Place Chatbot · 70 points



A.I.D.A. opens by asking which language, inside a working clone of the IRD Anguilla site.

THE PROBLEM

“Not everybody is excited when they hear the word taxes ... how can we make [it] easier, more welcoming, and more human?”

ISLAND WAVES / A.I.D.A.

Nobody is pleased to hear the word taxes. The brief from Anguilla's Inland Revenue Department was blunter than that — when the only tax officer is busy, who picks up? The team's answer was not a better phone queue. It was a tax-time sidekick that never goes to lunch.

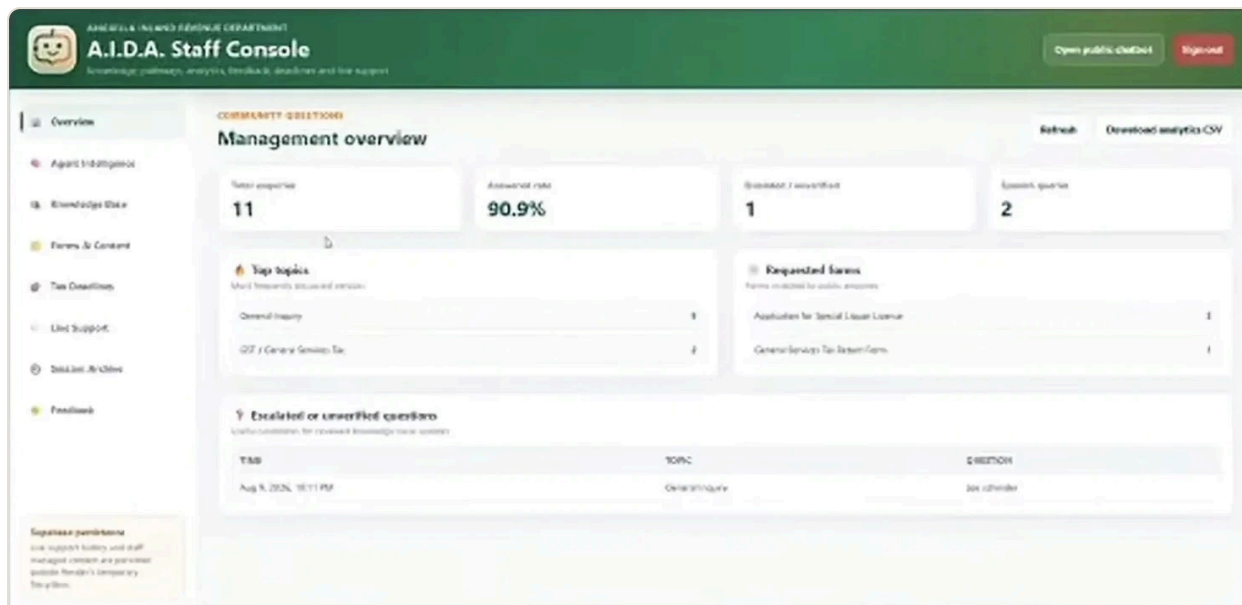
Anguilla's Inland Revenue Department published a brief with a staffing problem inside it. When the only tax officer is busy, the queue does not go away — it waits, and it grows, and the person at the end of it gives up and comes back tomorrow.

Island Waves was a joint team, drawn from Montserrat, Anguilla and Nevis, which meant they were building for a department most of them had never walked into. Hadassah Williams led the presentation.

WHAT THEY BUILT

A.I.D.A. runs inside a working clone of the official website of Anguilla's Inland Revenue Department (IRD), built so the team could prove embedded-widget deployment without touching the live platform. She opens by asking which language — English, Spanish or Mandarin — then answers the question everyone actually asks, in the dialect they ask it in, with a structured Universal Social Levy estimator: targeted follow-ups, an itemised estimate and a prominent legal disclaimer. She gives turn-by-turn directions to the IRD building and delivers fillable PDF returns inside the chat. Behind her sits a staff console the team nicknamed IRD's Kitchen — live monitoring, analytics, knowledge editing and human takeover.

3 languages English, Spanish and Mandarin	70 points, second place Chatbot Division
2 interfaces the public assistant and IRD's Kitchen staff console	24/7 availability against a department with one tax officer



IRD's Kitchen — the staff console behind **A.I.D.A.**, showing live enquiries, top topics and anything escalated.

WHAT THE JUDGES PRESSED ON

Dyson Lau, a Technical Officer at the Caribbean Institute for Meteorology & Hydrology (CIMH), asked two questions.

"Was the demonstrated website an existing site or built by you?" Built by them: a working clone of the IRD Anguilla website, made to prove embedded-widget deployment without touching the live platform.

"Do you see a commercial opportunity in fine-tuning regional dialect voice models?"

THE TURNING POINT

They did. The team had used Gemini and Google AI Studio prompting to tune pitch and inflection until A.I.D.A. sounded authentically Anguillian — and argued that licensing Caribbean-accented voice models is a real commercial opportunity, not a nicety.

“
Boy, how much taxes me got pay now?”

ISLAND WAVES / A.I.D.A.

BUILT WITH

Gemini

Google AI Studio

RAG

THE TEAM

Hadassah Williams — Lead Presenter

Michaela Conner — camper

Zenia Francis — camper

Kai Gray — camper

Khan Gray — camper

Chayzan Green — camper

Hallie-Jaiz Harris — camper

They did not fight the culture around tax. They spoke its language, down to the dialect of the question everyone was already asking.

Our reading of the build

An independent assessment by the editors of this publication — not the judges' scoring, and not the team's own account.

Craft

Building a clone of the client's own website to prove embedded deployment is the move of a team thinking about handover, not applause. It answers the question every client asks second — *how would this actually reach our users?* — before anyone has to ask it. The estimator is the other tell: it could have been a chat reply, and instead it is a structured flow with follow-ups, an itemised breakdown and a disclaimer where a disclaimer legally belongs.

What they learnt, and where it shows

Grounding is the thread. A tax estimate is the most consequential thing any bot at this showcase produced — get it wrong and someone underpays a government — and the team treated it accordingly, wrapping the estimator in mandatory disclaimers rather than letting a model speak freely about money owed. IRD's Kitchen is the human-handoff lesson made into furniture: the escalation path is not a promise in a slide, it is a screen somebody sits at.

Usability

Asking which language first, before anything else, is the right order for a trilingual population, and answering dialect in dialect is the single warmest decision in the whole field. The weakness is density: the staff console shows enquiries, answer rate, escalations and top topics on one screen, which serves an analyst better than it serves the officer with four minutes between callers. A first-run view of *what needs me now* would carry more of the load.

Scalability

Language coverage is the strength and the cost. Three languages is three knowledge bases to keep in step, and a mistranslated tax rule is worse than no translation — so the maintenance burden here is heavier than the feature list suggests. The widget architecture, though, is genuinely portable: any revenue department with a website and a document set could run this shape, which is what makes it interesting beyond Anguilla.

“ You've seen the customer side; now welcome to
IRD's kitchen.

ISLAND WAVES / A.I.D.A.



Read the A.I.D.A. chapter

Scan to open this chapter on your phone.

[eccuaicamp2026.netlify.app/islands/en/bo
ok/aida-island-waves/](https://eccuaicamp2026.netlify.app/islands/en/bo
ok/aida-island-waves/)

Images captured at the 2026 ECCU Chatbot & Multimedia Showcase, 28 August 2026.
Screen captures have been cropped to remove browser and streaming interface; the
chatbot interfaces and the presenters are unaltered.

The Golden Record

Topic · Question · Hard Fact · Source URL · Last Verified.

Ask a chatbot what a ferry costs and it will tell you. Ask it where that number came from and, in most cases, it cannot say — not because the answer is hidden, but because there was never an answer to hide. The figure was generated, not retrieved.

For a consumer product that is an irritation. For an organisation, it is the whole problem. A port cannot publish a tariff it cannot source. A revenue department cannot state a rate nobody signed off. The question *who says so?* is not pedantry — it is the difference between a service and a liability.

So the camp's first rule was not about Python. It was this: **a fact must be sourced before the bot may say it.**

Classify before you answer

Day eight gave the rule a name. `classify_authority()` — a function whose only job was to ask, of any claim, who is entitled to make it.

That turned out to be three questions, not one, and Team Code X gave the shape an acronym the camp kept: **ART**.

Authority — who is entitled to say this? An operator's official site is an authority. A travel blog is not. Neither is the model's own memory, however fluent it sounds.

Register — which record holds it, and in which voice it should be said? The word does double duty, deliberately. There is the record sense: tariffs live in one table, terminals and operators in another, cargo clearance in a third, and a question answered from the wrong one produces something confident and irrelevant. And there is the sense a linguist would recognise — register as *how you speak*. The same hard fact put to a port operator and to a nervous first-time traveller is one fact in two registers. The bot's persona is not decoration sitting on top of the answer; it is the second half of classifying it.

Territory — whose answer is it? This is the one that catches people, and in an eight-territory programme it caught everybody. The ferry terminal in Basseterre belongs to the St Christopher Air & Sea Ports Authority (SCASPA). Ask the same question about Nevis and the answer changes hands entirely — same question, same register, different island, different fact.

Get all three right and you may speak. Miss one and you are guessing in a way that sounds like knowing.

The Golden Record

By day ten ART had become a table, and the table had a name: the **Golden Record** — a Single Truth Table with five columns.

Topic. Question. Hard Fact. Source URL. **Last Verified.**

The first four are what you would expect of any knowledge base. The fifth is the one that shows real maturity, because it admits something most systems will not: facts decay. A tariff correct in July may be wrong in November. A terminal changes operator. A rate is revised.

A system that tells you when it last checked is more honest than one that simply sounds confident — and it is the only kind an organisation can defend.

Team ASPIRE pushed that second sense of Register furthest. Skye answers a six-year-old, Kaleb a ten-year-old, Zion a teenager — three registers over a fact that does not move an inch. What the EC\$1,000 is, where half of it sits, what a share actually means: identical in all three mouths, grounded against the same Ministry circulars. Only the register changes.

And a second dial sits on top of the guide. Zion can answer as The Professor or as The Llama — *the guide sets the learning level; the personality sets the tone*. Two independent axes, both of them register, both riding on a fact pinned before either was applied.

Which is exactly why it holds. Varying how you say something is only safe once you are certain what you are saying; do it the other way round and you are not adapting, you are improvising in three voices. Pin the fact, then choose the register. In that order.

Where guardrails come in

A Golden Record tells a bot what it may say. **Guardrails** are what happen when it should not.

The two are halves of one mechanism. Grounding supplies the permission; guardrails enforce the refusal. Without the second, the first is a suggestion — and week two had already demonstrated what a language model does with a suggestion.

So the rules were written as code rather than hoped for in a prompt. If no source exists, do not answer. If the source is too old, say so, or hand off. If the question touches security, customs or someone's private records, refuse and escalate to a human. Day nine's jailbreak lab existed to test exactly this: campers spent a session trying to talk their own bots past their own rules, on the principle that you should *think like a hacker, but build your chatbots like a defender*.

A guardrail is not a filter on rude words. It is a rule about when a system is entitled to speak.

Try it yourself

Below is a real question, answered by a real bot at the showcase. Place it with ART, watch the Golden Record row assemble — then move the *Last Verified* date and see what the guardrail will and will not permit.

Interactive element — open this chapter online to try it.

What it bought

This is the machinery underneath every build in this book.

W.E.S refusing to return a balance without a valid token. **PILOT** showing its sources with the date each was last checked, and the line on screen stating that a date is when a fact was verified, not a promise it is still true. **A.I.D.A.** wrapping a tax estimate in a mandatory disclaimer. **ASPIRE AI** grounded strictly against Ministry circulars rather than the open internet.

None of that is a feature a fifteen-year-old thinks to add. It is what you build when a real client's name is going on the output — and it started with a five-column table and a question about who says so.

From Chatbot to Crew

Six components, then guardrails, then grounding, then a crew of agents.

Day one had taken a chatbot apart on a whiteboard and named the pieces: interface, brain, memory, personality, decision pathways, tools. Six components, so that nobody in the room could use the word *chatbot* without knowing what was inside one.

The bot is the GPS, T. Valerie Onu told them that first morning. *The human is the driver*.

Everything since had been built on that frame. A dictionary bot that matched a question to a stored answer, and the quick discovery of how little that could carry. Guardrails, so it would refuse rather than improvise. The Golden Record, so every fact arrived with a source and a date attached. Retrieval, so the answer came out of a document the client owned rather than the model's memory of the internet.

By the start of week four the campers had a good bot. It had one shape, though, and the shape never varied: a question arrives, the system looks something up, an answer goes back.

One turn. One decision. One responsibility.

Real work does not arrive in that shape.

Before the AI, the workflow

The next move was not technical at all.

Campers were asked to look at ordinary, tedious business processes — the kind everybody has queued inside at some point — and take them apart as *workflows*. Not "applying for a loan" as a single event, but as a sequence: who touches it, in what order, what each person needs in hand before they can act, and what happens when the thing they need is not there.

That last question turned out to be the entire subject.

They practised on simulations before going anywhere near a client's problem, and the simulations were deliberately playful — issue a festival pass, walk an application end to end, step the sequence forward and watch what accumulates on the shared clipboard as

it moves. The value of a simulation is that you can run it again, and break it on purpose. Nobody gets to do that with a bank.

Breaking it on purpose is where the learning actually sat. A workflow that only ever runs smoothly teaches nothing, because every real process spends most of its life in the exceptions. So each simulation shipped with its failures built in: the festival pass where the age proof is missing. The payment that fails. The identity document too blurry to read.

Only then did automation arrive — and it arrived as a question rather than a tool. Which of these steps could a system take? Which must a human keep? And what has to happen at the exact moment a step cannot complete?

Tariq is where that question got its worked example.

Tariq wants a loan

That is the whole premise, and it is deliberately ordinary. He walks into a bank — or opens a chat window, which by 2026 is the same thing — and asks for money to grow a business. What happens next is not one question with one answer. It is a sequence, and every step in it belongs to somebody different.

Somebody has to greet him and work out what he actually needs. Somebody has to establish who he is and what kind of customer he is. Somebody has to look up what the bank's own rules say about lending. Somebody has to check those rules against his documents. If something is missing, somebody has to go back and ask him for it — which means the sequence has to be able to *turn around*. Somebody has to inspect the result before it goes out. And somebody has to hand it back to Tariq in language he can use.

Ask a single chatbot to do all of that and it will try. It will also, somewhere in the middle, quietly invent a lending rule.

Why we taught it as a crew

This is the moment the camp stops teaching generative AI chatbots and starts teaching Agentic AI engineering.

A crew is not a chatbot with more prompts. It is the same problem broken into named jobs, each with its own inputs, its own constraints and its own right to refuse. **Tariq's Digital Crew** gave those jobs faces: Grace greets, Marcus profiles, Ava retrieves what the rules

say, Daniel checks compliance, Maya interviews when something is missing, Theo inspects, Zara hands the outcome back.

Seven agents, one shared clipboard, and — the part that matters most — an edge that loops backwards. When Daniel finds a document missing, the work does not fail and it does not continue. It returns to Maya, who asks Tariq for the thing that is missing, and only then moves on.

Told as a story first

None of that lands as an architecture diagram on a Tuesday morning.

So it was taught as a story. The camp built **comics** for the agents, giving each one a character before it was ever a function, and produced a video walking through Tariq's journey from the request to the answer. Campers met Grace and Daniel and Maya as people with jobs before they met them as nodes in a graph.

That order was the point. A fifteen-year-old who has followed Tariq's application through seven pairs of hands already understands handoffs, state and conditional routing — they simply do not yet know those are the words. When the code arrives, it is not a new idea. It is their idea, written down in Python.



Watch: Tariq's Digital Crew

Scan to watch.

youtu.be/1zI0fdKrxIA

Then the hard part

Storytelling gets you to the idea. It does not get you to a production system, and the camp did not pretend otherwise.

The framework the campers were taken to was **LangGraph**, and it is worth being precise about what that means, because it is easy to assume a summer programme reaches for the gentlest tool available. It did the opposite.

LangGraph is a low-level framework. In software engineering, low-level means granular control and more explicit code — you are not handed abstractions that do the thinking for you. Where a high-level framework lets a developer declare an agent's role in plain

English and trust the model to work out the steps, LangGraph requires three things to be written by hand:

Nodes — the precise code blocks or model calls that execute an action.

Edges — the conditional logic and decision paths connecting those actions, including the ones that loop backwards.

State — a central schema recording exactly what the crew knows at any given moment.

Because it hides none of that behind convenient abstractions, it gives absolute control over loops, self-correction and human-in-the-loop intervention. That control is the entire reason enterprises choose it. When a hallucinating agent could cost a bank money, you want a state machine with strict boundaries, not an autopilot that may wander.

Why low-level, specifically

The camp could have reached for a high-level framework. It would have been faster, gentler, and produced something demonstrable by the afternoon. It chose not to, and the reason was security.

In a high-level framework the model decides what happens next. That is the selling point — describe a role, and the system works out the steps. It is also, precisely, the attack surface.

Prompt injection works by getting text into a model's context that the model treats as instruction. If the model is only writing a reply, a successful injection produces a bad reply. If the model is also choosing the route, a successful injection chooses the route: skip the compliance check, reach a tool it was never meant to touch, return a figure nobody verified. One manipulated instruction, and the blast radius is the whole workflow.

Write the graph explicitly and that class of attack has nowhere to go. An injection can still corrupt what a node *says*. It cannot invent an edge that does not exist. It cannot route around Daniel, because Daniel is not a suggestion — he is a node the traffic passes through. It cannot call a tool the graph never wired up. The model's authority stops at the boundary of the node it is running inside.

That is what hardening means here. Not a filter bolted on afterwards, but an architecture in which the dangerous paths were never built.

It is also what made the day-nine lab worth doing. Campers spent a session trying to jailbreak their own bots, and that exercise only teaches something if there is a structure to test against. Break an autopilot and you have proved a model can be talked into things, which everybody already knew. Break a coded graph and you have found out exactly which node leaked and precisely what it was permitted to reach — a finding narrow enough to fix.

And it is why CUB-VA could state its invariant as flatly as it did: that it will never request, store or reveal confidential banking credentials — no account numbers, no passwords, no balances. An agent that decides its own next step cannot honestly promise that. An agent that only walks paths somebody wrote can.

What that actually asks of a learner

LangGraph is a direct implementation of graph theory and state machines applied to generative AI. Using it well calls on directed and cyclic graphs, data persistence and asynchronous programming — computer-science fundamentals, not chatbot tips. It is a staple of advanced university courses and production-grade AI workshops, and it is generally considered too advanced for someone building their first prototype.

It was taught to Caribbean teenagers in week four.

Where the field actually is

It is worth being clear about how recent all of this is. Retrieval and agent orchestration are not settled practice with a decade of textbooks behind them. They are the live problem in applied AI — being worked out right now, in production, by teams considerably better resourced than a summer programme.

The reason is that the model was never the hard part. Any organisation can call an application programming interface (API). What separates a demonstration from something a bank will put its name on is everything arranged around the model: where the facts come from, what the system is permitted to say, what happens when a step fails, and who it escalates to when it should not answer at all. That is orchestration, and it is where the genuine difficulty currently sits.

Which is the claim worth making about this programme, and it is a narrow one. The campers were not handed last decade's technology repackaged for schools. They were taken to the part of the field that is still being argued about — and asked to ship something in it.

Interactive element — open this chapter online to try it.

The lab the campers worked through is public, and still is. It builds a retrieval agent line by line — what you type on one side, what LangGraph supplies on the other, and how each step maps onto the agent organisation you drew before you wrote any code.



The Agentic RAG build-along lab

Scan to open on your phone.

eccuaicamp2026.netlify.app/labs/agentic-rag

What the campers came away with

So let's strip out the chatbot framing and look at what the twenty days actually transferred.

Graph theory, applied. Nodes, edges, directed and cyclic flow — not as an abstraction but as the thing deciding whether Tariq's application loops back or proceeds.

State design. A schema for what a system knows at any given moment, and the discipline of reasoning about what happens when part of it is missing.

Control flow under uncertainty. When to retry, when to halt, when to stop and hand to a human. Most software assumes it will succeed; agents have to be written by someone who assumes they will not.

Retrieval over a governed corpus. Grounding an answer in a source somebody owns, and citing it.

Adversarial testing. Day nine was campers attacking their own systems on the principle that you should think like a hacker and build like a defender.

Output validation. Schema checkers, refusal conditions, and the habit of not trusting a model's output because it reads well.

Deployment. FastAPI to GitHub to Render, and a start command that has to be exactly right.

And Python, to an externally examined standard.

The first seven are the ones a hiring manager would recognise as senior. The last is the one that fits on a certificate. What is not on any list is the judgement underneath all of it — knowing when a system should refuse — and that is the part you cannot look up.

The transfer

Rigour of that kind only counts if it survives contact with a real client, and this is where the twenty days can be audited rather than described.

W.E.S routes an emergency before it collects a name — a conditional edge, taken because a flooded yard cannot wait behind an identity check. **A.I.D.A.** escalates to a staff console rather than improvise a ruling on tax — a human-in-the-loop node, placed exactly where being wrong costs most. **ASPIRE AI** added output-schema checkers after its agents got tangled in multi-turn testing, which is a state problem diagnosed and repaired by the people who caused it. **PILOT** hands anything touching customs to a port dispatcher — a halt condition, written deliberately rather than hoped for.

Those are not features anyone thought to request. They are what a graph looks like when the person drawing it has understood that the hardest part of an agent is knowing when it must stop.

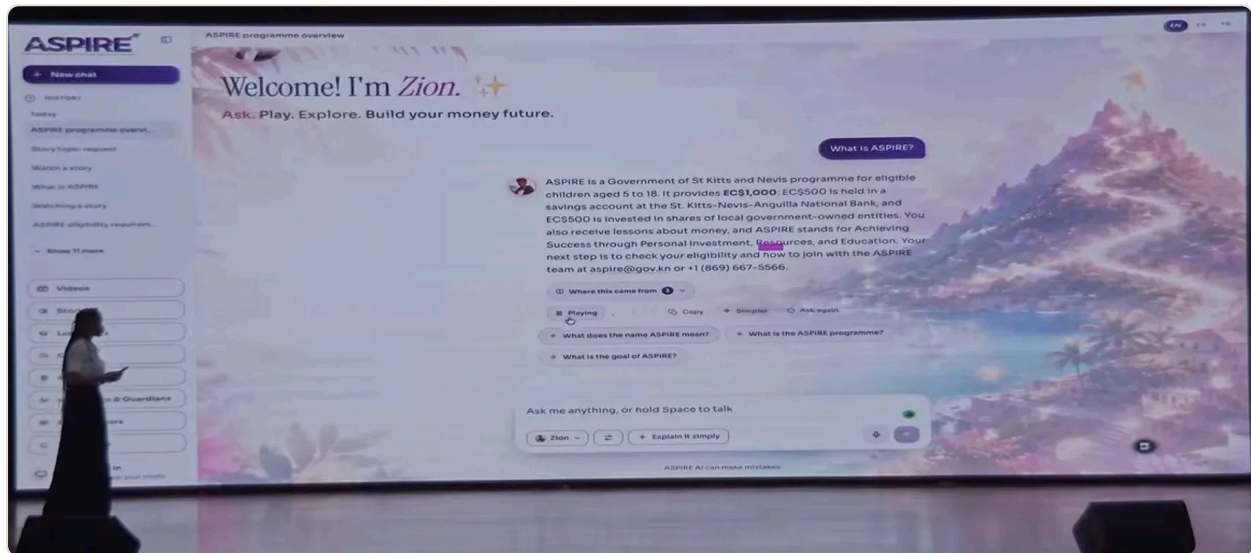
On stage, watching their own system hand off correctly under pressure, one team put it more plainly than any of this: *That is what agentic means in practice!*

A Bot That Changes Who It Is

THIRD PLACE · TEAM ASPIRE × THE ASPIRE PROGRAMME

Team ASPIRE · ASPIRE Programme · SKN

🏆 3rd Place Chatbot · 68 points



Rayanna Morris-Greenidge presents Zion, the ASPIRE guide for ages thirteen and up.

THE PROBLEM

“It shouldn't talk to a 6-year-old the way it would talk to a 17-year-old.

ASPIRE AI

The Government of St Kitts and Nevis gives eligible young people an EC\$1,000 financial start through the ASPIRE Programme (Achieving Success through Personal Investment, Resources and Education). Knowing that and understanding it are different things, and the gap between them is where a good scheme quietly fails. A six-year-old asking about ASPIRE and a seventeen-year-old asking about ASPIRE need two different conversations.

ASPIRE stands for Achieving Success through Personal Investment, Resources and Education. Of the EC\$1,000 a young person receives, half is held in a savings account at the St Kitts-Nevis-Anguilla National Bank and half is invested in shares of local government-owned entities.

That is a lot of financial machinery to explain to someone who has just heard that the government is giving out money.

WHAT THEY BUILT

A multi-agent system with age-tiered guides, because it should not talk to a six-year-old the way it talks to a seventeen-year-old. Skye handles ages five to eight, where money is seeds that help children grow. Kaleb takes nine to twelve — saving, deposits, interest. Zion takes thirteen and up, into shares, eligibility and how the programme actually works. On top of the guide sits a tone engine: Zion can answer as The Professor, formal and precise, or as The Llama, colloquial and loose. Language switches live, mid-conversation, with context preserved, and the whole system is grounded strictly against Ministry of Finance circulars and ASPIRE statutes rather than the open internet.

3 age-tiered guides Skye (5–8), Kaleb (9–12), Zion (13+)	2 tones per guide The Professor and The Llama
68 points, third place Chatbot Division	EC\$1,000 the grant explained per eligible young person

WHAT THE JUDGES PRESSED ON

Dr David Farrell, the panel's chief judge, asked two questions.

"Is Aspire AI portable to other Eastern Caribbean territories launching youth wealth funds?" Yes, the team answered — the design is modular: swap the jurisdiction's documents, keep the age tiers.

"You utilized OpenAI, Claude, and Hugging Face rather than just Gemini. How do you govern multi-agent pipelines to avoid rogue behaviour?"

THE TURNING POINT

The team did not deflect. In multi-turn testing their agents had got tangled up in one another; they fixed it with rigid system prompts and output-schema checkers, and said so on stage.

“
The guide sets the learning level; the
personality sets the tone.

ASPIRE AI

BUILT WITH

OpenAI

Anthropic Claude

Hugging Face

RAG

THE TEAM

Dre Martin — Coder, Presenter

J'Nya Jack — Designer, Presenter

Rayanna Morris-Greenidge — Team Leader, Designer, Presenter

Financial literacy fails when it talks down to you or
over your head — so they built a bot that changes who it
is depending on who is asking.

EDITORIAL REVIEW

Our reading of the build

An independent assessment by the editors of this publication — not the judges' scoring,
and not the team's own account.

Craft

Age-tiering is easy to claim and hard to do. Most attempts amount to shorter
sentences for younger users. This one changes the metaphor — money as seeds

at five, deposits and interest at nine, shares and eligibility at thirteen — which means somebody thought about what a child can actually hold in their head at each stage rather than just adjusting reading level. Separating the guide from the tone is the sharper piece of design: two independent dials instead of six hard-coded personalities.

What they learnt, and where it shows

This is the clearest agentic build in the field, and also the one that hit the wall the curriculum warned about. Multi-agent orchestration was taught in week four; agents getting tangled in multi-turn conversation is precisely the failure that orchestration invites. The fix they reached for — rigid system prompts plus output-schema checkers — is the guardrails week and the structured-output discipline applied to a problem they discovered themselves, which is a better proof of learning than a working demo.

Usability

Live language switching with context preserved is genuinely difficult and largely invisible when it works, which is the mark of good engineering. The open question is discovery: a child does not know that Skye, Kaleb and Zion exist, or that a tone dial is available. Age-tiering only helps if the right tier is reached, and the routing between them deserves as much design attention as the tiers themselves.

Scalability

The team gave the honest answer on portability under questioning — swap the jurisdiction's documents, keep the age tiers — and it is the right one. The architecture is the transferable asset here, more than the content. The constraint is that three guides across multiple languages multiplies the surface being grounded, and every one of those combinations is a place where a statute could be quoted at the wrong reading age. Three models in the stack also means three sets of costs and three failure modes to monitor.



Read the ASPIRE AI chapter

Scan to open this chapter on your phone.

[eccuaicamp2026.netlify.app/islands/en/bo-
ok/aspire-ai/](https://eccuaicamp2026.netlify.app/islands/en/bo-
ok/aspire-ai/)

Images captured at the 2026 ECCU Chatbot & Multimedia Showcase, 28 August 2026.
Screen captures have been cropped to remove browser and streaming interface; the
chatbot interfaces and the presenters are unaltered.

Ship It Live

A live deploy failed on a wrong start command, and was fixed on stage.

Day thirteen was the deploy. FastAPI to GitHub to Render, live, in front of the cohort.

It failed.

My repo failed, said Nicole Johnson, the Python lead, in the middle of it. Then, having found it: *your start command is important*.

The command was wrong. That is all it was — the kind of error that produces a wall of red output and tells you almost nothing about the actual cause, and which every engineer who has ever deployed anything has lost an afternoon to. She found it in front of everyone, fixed it in front of everyone, and said the line the day is remembered for: *my little bot is online. And it is ready to go*.

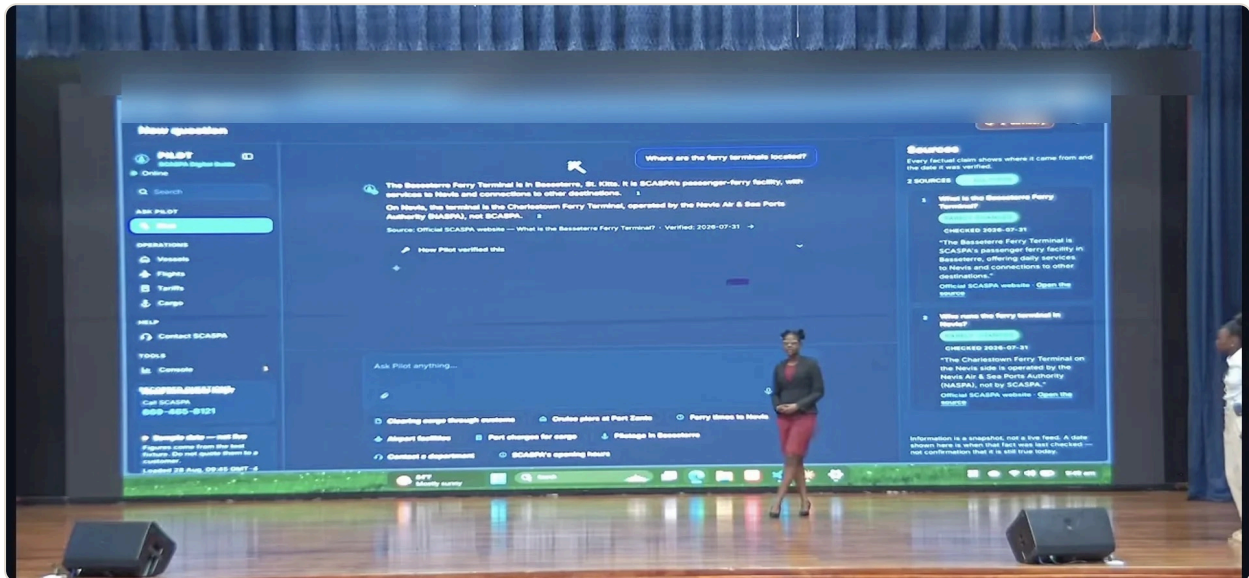
The teaching value of that failure exceeded the teaching value of a clean deploy by some distance. A demonstration that works shows you a happy path. A demonstration that breaks and is repaired shows you what the work actually consists of — which is mostly this, and which is survivable.

Chief Judge Farrell said much the same thing from the stage at the showcase, three weeks later: technical setbacks are standard, and feedback is fuel. By then several teams had already lived it.

The Port That Never Sleeps

TEAM SCASPA · ST KITTS & NEVIS

Team SCASPA · St Christopher Air & Sea Ports Authority · SKN



Nahamani Carey presents PILOT, its Sources panel showing where each ferry-terminal fact came from and when it was last checked.

THE PROBLEM

You are travelling from St Kitts to Nevis. You want the ferry schedule, or the port tariff, or what it will cost to clear your cargo. So you call the St Christopher Air & Sea Ports Authority (SCASPA). And you wait. Then you wait some more. The published brief put it plainly — every plane, ship and cruise passenger, and the phones never stop.

SCASPA is the St Christopher Air and Sea Ports Authority: the machinery that moves everything and everyone between two islands. Ferries, cargo, cruise passengers, and the airport that carries whatever does not go by sea.

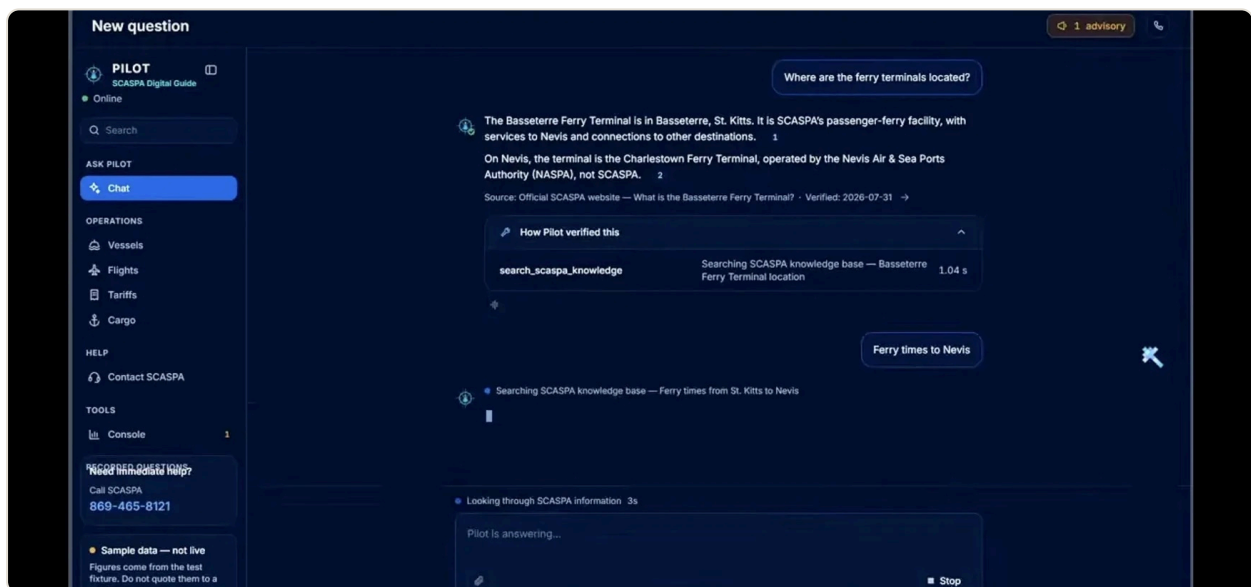
The published brief did not describe a technology problem. It described a telephone that never stops ringing — and behind each of those calls, someone standing at a terminal without the one piece of information they needed before they left home.

WHAT THEY BUILT

A 24/7 self-service guide over verified SCASPA data: inter-island ferry schedules, maritime tariffs, cargo clearance documentation and Robert L. Bradshaw International Airport logistics. Every factual claim carries a Sources panel showing where it came from

and the date it was last checked, and the interface says plainly that information is a snapshot rather than a live feed. Anything touching security or customs clearance is handed to an on-duty port dispatcher rather than answered.

1.04s verification time knowledge-base search behind a live answer	4 operations covered vessels, flights, tariffs and cargo
2 sources shown per factual claim, each with a checked date	Dispatcher handoff for anything needing security or customs clearance



PILOT shows its working — the knowledge-base search behind the answer, timed at 1.04 seconds.

BUILT WITH

Python

RAG

Dispatcher handoff

THE TEAM

Josiah Benjamin — Group Lead, Coder

Nahamani Carey — Designer, Presenter

Jernyah Mills — Presenter, Designer

A port is a lifeline, not a help desk — and the bot's job is to answer instantly while knowing exactly when to pass a traveller to a human.

EDITORIAL REVIEW

Our reading of the build

An independent assessment by the editors of this publication — not the judges' scoring, and not the team's own account.

Craft

PILOT is the most disciplined build in the field about what it does not know. The Sources panel is not decoration — it carries the claim, the origin and the date it was last checked, and the interface states outright that a date is when a fact was verified rather than a guarantee it is still true. A team that ships *information is a snapshot, not a live feed* on the screen has understood something about public-sector data that a lot of production software still has not.

What they learnt, and where it shows

The Golden Record is the whole chapter here. Topic, question, hard fact, source, last verified — the teaching artefact and the shipped interface are close to the same object. The handoff discipline comes from the same place: security and customs questions are not answered badly, they are not answered at all, which is the authority-classification lesson enforced as a rule rather than a hope.

Usability

Showing the tool call — the knowledge-base search, timed — turns a loading state into an explanation, and it is the reason a sceptical traveller might believe the answer. The tension is that the Sources panel occupies roughly a third of the screen for a product whose users are frequently standing at a terminal on a phone. What reassures a judge in a hall may crowd the person it was built for.

Scalability

The dispatcher handoff is the honest constraint: it scales exactly as far as there is a dispatcher on duty, so out-of-hours the bot's most important capability is the one that stops working. The demonstration also ran on sample data drawn from a test fixture, and the interface said so — creditable in a presentation, but it means the live data path is the piece still to be proven. The domain coverage, though, is unusually well-bounded, and a well-bounded scope is the easiest kind to keep accurate.



Read the Pilot chapter

Scan to open this chapter on your phone.

[eccuaicamp2026.netlify.app/islands/en/bo
ok/scaspa-pilot/](https://eccuaicamp2026.netlify.app/islands/en/bo
ok/scaspa-pilot/)

Images captured at the 2026 ECCU Chatbot & Multimedia Showcase, 28 August 2026.
Screen captures have been cropped to remove browser and streaming interface; the
chatbot interfaces and the presenters are unaltered.

The Chatbot Showcase Field

All eleven chatbot presentations.

Eleven chatbots reached the showcase floor. They are the Chatbot Division's share of the eighteen solutions presented that day — the other seven were multimedia pieces, and belong to a companion edition.

These eleven are distinct from the twenty-six Agile Pods the programme began with. Pods consolidated into teams as the build progressed, which is why the two numbers differ and why both are correct.

TEAM	TERRITORY	CLIENT	BOT	WHAT IT DID
AquaCoders 	SVG	NAWASA (Grenada)	WES	73 points. OCR meter-reading, token-locked billing
Island Waves 	Montserrat / Anguilla / Nevis	IRD Anguilla	AIDA	70 points. Trilingual, USL estimator, IRD's Kitchen
Team ASPIRE 	St Kitts & Nevis	ASPIRE	Aspire AI	68 points. Age tiers Skye, Kaleb and Zion; Professor and Llama tones
Team Grenada	Grenada	NIS Grenada	NISA	Next.js and FastAPI with LangGraph; Source Toggle; personas Ben and Eliana
Team SVG	SVG	NAWASA (Grenada)	Aquasist	Accessibility, territory select, defect reporting
AI Dominators	Dominica	Caribbean Union Bank	CUB-VA	Persona Dawn Roberts; voice, calculators, zero retention
Team Code X	St Lucia	NIS Grenada	NISSI	The ART framework — Authority, Register, Territory
Team Trackstar	SVG	IRD Grenada	TESSA	Rental-income tax guidance; FastAPI and LangGraph
Team SKN	St Kitts & Nevis	ACB Caribbean	ACB Digital Assistant	Card freeze, mortgage triage, five languages
Team SCASPA	St Kitts & Nevis	SCASPA	Pilot	Ferry, tariff and cargo guidance; dispatcher handoff
Team A&B	Antigua & Barbuda	LUCELEC	<i>(unnamed)</i>	Appliance cost estimator, load-shed schedules

Two things in that table look like errors and are not. Two chatbots were built for the National Water and Sewerage Authority (NAWASA) — **W.E.S** and Aquasist — and two for Grenada's National Insurance Scheme (NIS), NISA and NISSI. Different teams, different territories, the same client brief approached twice. Reading them side by side is one of the more instructive things this book offers.

Banking, two ways

Two teams built for banks, and arrived somewhere quite different.

CUB-VA, built by AI Dominators in Dominica for the Caribbean Union Bank (CUB), was framed commercially. Its hook persona is Dawn Roberts, a forty-six-year-old equipment-rental owner who cannot afford forty minutes on hold. In the live demonstration it parsed *managing my shares* and *a business loan* in a single breath, distinguishing term loans from commercial lines of credit. It offered hands-free voice-to-voice, one-tap tools for branch hours, calculators and an ATM locator, English, French and Spanish, and a high-contrast mode. Its security invariant was stated plainly: it will never request, store or reveal confidential banking credentials — no account numbers, no passwords, no balance figures. Pressed by Dr Farrell, the team conceded that early edge cases had produced generic model answers, and that the fix was ingesting CUB's own policy circulars. Sessions do not persist.

The ACB Digital Assistant, from Team SKN, was framed for retail and after hours — *like having a personal banker in your pocket 24/7, then instant, intuitive, and available before the branch even opens its doors, with no nested menus*. It demonstrated an authenticated greeting, an instant lost-card freeze that acted rather than explaining, native Spanish, French, Dutch and Mandarin, and mortgage triage that pre-qualifies before routing to an advisor. On logging, the team was straightforward: the pipeline is designed, the dashboard is not yet live.

One team built for the entrepreneur. The other built for the customer at eleven at night. Both were reading the same industry.

Honest Edges

Eight edges, owned in public — and why they are the programme, not its failures.

This is the spine of the book, and the reason the programme is worth writing about.

A camp that reports only its successes teaches its participants that success is the only thing worth reporting. In a region where we are taught to hide our drafts and only ever display the polished final product, this one kept a fierce record of its edges:

The curriculum flip — abandoning the rigid dictation of Python syntax in favour of reverse-engineering from a finished product.

Sample code seeded with deliberate errors, forcing campers to wrestle with the engine rather than just watch it run.

The day-six re-teach, when a simple quiz revealed the cohort did not understand memory, and a facilitator stopped the entire schedule to anchor them.

The day-thirteen live deploy that crashed on a wrong start command and was fixed live on stage.

The ElevenLabs credit blowout — voice features intentionally left on the bench rather than faked with a recording.

The multi-agent tangle — six distinct AI guides looping into one another's turns during multi-turn testing, fixed with rigid prompts and schema checkers, then owned out loud under direct questioning from the chief judge.

The location edge-case, held back from production rather than shipped broken for a quick win.

The final presentation — offered the safety of a pristine slide deck, choosing instead to risk raw, unvarnished Python in front of a room of visiting ambassadors.

None of these are failures of the programme. They are the programme. A young camper who has debugged a broken system live in front of a room full of peers and dignitaries builds a completely different relationship with difficulty.

They move past the fear of being wrong, straight into the mechanics of making it right.

In our classrooms, we are often trained to chase the perfect mark, to fear the red ink, and to perform certainty even when the foundation is shaking. But software does not care about manners or prestige; it only cares about truth.

Interactive element — open this chapter online to try it.

Failing Forward

Failing forward is not a placeholder for a participation trophy — it is the literal engineering loop that improves the final product.

You cannot build a bulletproof system without first seeing where it shatters. By forcing these youth campers to own their glitches on stage, analyse the wreckage, and iterate on the fly, the programme achieved superior technical results while building something far deeper. It forged a generation capable of standing in the middle of a broken system, clear-eyed and unashamed, ready to build the fix.

The Showcase

Panel, partnership, results.



The title card for the 2026 ECCU Chatbot & Multimedia Showcase and Closing Ceremony, as streamed on ECCB Connects.

Image: Eastern Caribbean Central Bank (ECCB) · [the ECCB Connects livestream of the showcase](https://www.youtube.com/watch?v=OqICcUGcNFk) (<https://www.youtube.com/watch?v=OqICcUGcNFk>), 28 August 2026.

Eastern Caribbean Central Bank (ECCB) headquarters, 28 August 2026, half past eight in the morning, streamed on ECCB Connects. Mrs Sybil Welsh, Senior Project Specialist at the ECCB and the camp's project manager, took the room. The opening prayer, by Mrs Francisca Cindy Dorius of St Lucia, was given in English and Mandarin.

The guest list read like a regional summit rather than a camp closing. H.E. Harriet Cross, Governor of Montserrat. Hon. Veronica Dorsett-Hector, Minister of Infrastructure, Utilities and Labour. H.E. Edward Tao, Ambassador of the Republic of China (Taiwan) to St Kitts and Nevis. H.E. Nicole Su, Ambassador to St Lucia, represented by Deputy Consul Ricardo Chu. Dr Tracy Polius, Chief Director of Policy at the ECCB. Beverly Mendez, Permanent Secretary for Education in Montserrat. Paul Maxwell of St Patrick's Co-operative Credit Union. Ms Fan of the Taiwan Embassy, and Roy Lo of the Taiwan Technical Mission. Governor Timothy Antoine was acknowledged in the vote of thanks.

The panel

Not, as the teams were carefully told, a panel of engineers — a regional technical panel. Dr David Farrell, Principal of the Caribbean Institute for Meteorology & Hydrology (CIMH), as chief judge. Dyson Lau and Horace Lashley, also of the Institute. Amani Richardson, a high-school IT teacher and President of the Anguilla National Youth Council. And Raywen, Project Manager of the Technical Mission of the Taiwan International Cooperation and Development Fund (TaiwanICDF) in St Lucia.

The bar

The questions were real ones. Vibe-coding against hand-written Python. What still fails. Prompt injection and rogue-agent defence. How the time savings were benchmarked.

The teams answered with engineering candour — edge-case bugs, agents that got tangled up, deployments waiting on client APIs — rather than with claims they could not support. Several of the strongest moments in this book are teams volunteering their own limitations to a judge who had not yet found them.



Watch the 2026 showcase

Scan to watch.

www.youtube.com/watch?v=OqICcUGcNFk

The close

The vote of thanks was given by Shannon of St Vincent and the Grenadines, team leader of the 2026 winning chatbot, **W.E.S.** Tribute was paid to Anna-Kay of St Lucia, the programme's certification architect. Then an all-hubs group photograph, and Farrell's send-off: setbacks are standard, and feedback is fuel.



EASTERN CARIBBEAN
CENTRAL BANK

ECCU Youth Harness AI Skills to Develop Solutions for Real-World Challenges

Young innovators from across the ECCU showcased AI-powered chatbots and multimedia projects at the closing ceremony of the 2026 ECCU Generative AI and Python Summer Camp.



From the ECCB's announcement of the 2026 showcase, with a photograph of campers and guests on stage at the closing ceremony.

Image: Eastern Caribbean Central Bank (ECCB) · [ECCU Youth Harness AI Skills to Develop Solutions for Real-World Challenges](https://www.eccb-centralbank.org/news/eccu-youth-harness-ai-skills-to-develop-solutions-for-real-world-challenges) (<https://www.eccb-centralbank.org/news/eccu-youth-harness-ai-skills-to-develop-solutions-for-real-world-challenges>), 1 September 2026.

Why This Works

Authentic youth engagement and the innovation ecosystem.

This is the chapter a funder reads to understand not what the camp made, but why the model matters.

Four ways of thinking

The camp taught Python, AI & Agentic Architecture, Engineering and Empathic Generative AI Chatbot Development. However, the deeper syllabus unpacked critical soft skills, four modes of thinking, and each is visible in the builds.

- **Critical thinking.** The first rule — a fact must be sourced before the bot may say it — is critical thinking made into engineering. It shows up in **W.E.S** refusing to invent a balance, in **A.I.D.A.**'s mandatory disclaimers, and in the day-nine lab where campers set out to jailbreak their own bots to test their system prompts. By the showcase, teenagers were fielding a judge's cross-examination on prompt injection.
- **Creative thinking.** A water bot introduced by a rap. A bank explained through Dawn Roberts, a forty-six-year-old equipment-rental owner who cannot afford forty minutes on hold. This is the A in STEAM — arts fused with engineering — and it is why the work connects with people rather than merely passing tests.
- **Design thinking.** Every team started from a person. Patrick with a flooded yard. Ben on a construction site. Tom waiting for a ferry. A six-year-old who cannot be spoken to like a seventeen-year-old. ASPIRE's age tiers and **W.E.S**'s adaptive avatars are design thinking made literal: the product changes shape for the user in front of it.
- **Systems thinking.** The jump from a single chatbot to an orchestrated crew of agents — LangGraph nodes handing off on a shared clipboard — is systems thinking. Campers learned to see the whole machine, not just the button. With the World Economic Forum projecting that 39% of all worker core skills will fundamentally change by 2030, this ability to navigate complexity is no longer optional. As a direct result of this shift, global enterprise data now places systems thinking firmly among the top 10 fastest-growing workforce capabilities. It has become the ultimate meta-skill for an automated world — empowering these youth to orchestrate interconnected networks and future-proof entire economies by shifting from short-term crisis management to restructuring the root causes of systemic instability.

Authentic engagement

Engagement here was not a worksheet dressed up as a game. It was authentic because the stakes were.

Campers did not imagine a bank; they built for one, and answered to it. Role rotation meant everybody wrote code, while the non-code seats gave campers who did not see themselves as programmers a way to contribute from strength. **A.I.D.A.** answering *Boy, how much taxes me got pay now?* in dialect is empowerment in a precise sense: the technology bent to the culture rather than the other way round.

And they were allowed to fail in public. A young person who has debugged live in front of ambassadors has a different sense of what they can do.

An ecosystem, not an event

VISIONARY PARTNERSHIP

This model of regional transformation exists because of the unwavering support and funding of the Eastern Caribbean Central Bank (ECCB) and the Republic of China (Taiwan) International Cooperation and Development Fund (ICDF), whose joint vision conceived this camp not as an event, but as an ongoing investment in our youth.

Every actor in this programme is a real institution. Eleven client organisations — banks, a port, water, tax, social security — briefed and evaluated the work. The region's central bank hosted and streamed the showcase. The ICDF, in its third consecutive year, and a technical panel from the Caribbean Institute for Meteorology & Hydrology (CIMH) connected Caribbean teenagers to a global technology and climate network. freeCodeCamp certification, portfolios and a defined progression from camper to Junior Consultant to agentic architect give the skills somewhere to go. And the whole thing is federated across eight territories, each able to run its own instance.

That is what ecosystem means. Not a single spark, but a circuit — youth, enterprise, bank, sponsor, educators and certification wired together so the energy keeps flowing after the camp ends.

Sources

World Economic Forum (2025). [The Future of Jobs Report 2025](#). Employers expect 39% of workers' core skills to change by 2030.

Lepaya (2026). [State of Skills 2026: 10 future-proof skills every workforce needs by 2030](#). Drawing on training data from 196 organisations worldwide, it ranks systems thinking among the top 10 fastest-growing workforce skills.

What Comes Next

The alumni pathway.

Prototypes are common. Minimum Viable Products (MVPs) are rare, and Enterprise Client Access is rarer still.

Every showcase solution addressed a brief from one of eleven real client organisations. The winning teams demonstrated working MVPs rather than static mock-ups: **W.E.S** with webhooks and REST endpoints code-complete and waiting only on the portal of the National Water and Sewerage Authority (NAWASA); **A.I.D.A.** running as an embedded widget inside a working clone of the website of Anguilla's Inland Revenue Department (IRD); **ASPIRE AI** running a live multi-model agent pipeline.

And the certification is earned rather than awarded — roughly three hundred hours of projects, with the certification projects requiring the whole curriculum to be applied rather than followed. Campers sat the official freeCodeCamp Python examination, and 50+ campers gained Python Certification — an externally verifiable global credential with a public certificate URL for each camper. That is the competency mandate in practice: a credential that demonstrates transferable skill and can be checked by anyone who cares to, rather than taken on trust.

The truth the numbers cannot hold

Access to these rooms is sacred. Having enterprise clients from the jump — rather than simulating them in an academic vacuum — permanently alters a young person's baseline of reality. A teenager who has briefed a bank, been cross-examined by a regional technical panel, and shipped a working product has had their trajectory changed. They no longer see themselves as students completing an assignment; they are digital producers delivering undeniable business-to-business (B2B) value. Doors, networks, and self-belief are now open that were closed the week before.

What happens now

Alumni pathways begin with Junior Consultants, scaling up to AI and agentic architects and leaders. But the reality of this ecosystem has already outpaced a standard alumni track: the client engagement is continuing.

Several participating enterprises have recognized these youth-built MVPs not as temporary camp projects, but as critical, ongoing operational concerns. Teams are currently in discussions to continue working with the institutions they built for. The implication here is massive: we are watching the real-time incubation of new, youth-led tech startups. By holding onto these enterprise relationships, these teenagers are transitioning from campers into tech founders and vendors.

Each territory is running its own instance, while clients carry their MVPs toward live deployment hand-in-hand with the young architects who engineered them.

The camp ends. The ecosystem it lit does not.

The Macro Impact: Digital Sovereignty and the Forward Deployed Engineer

Why the Eastern Caribbean's economic future depends on the blueprint built at this camp.

The ecosystem lit by this camp does not end with the showcase, nor is it simply a localized educational success story; it is the blueprint for regional digital sovereignty. As these youth step onto the alumni pathway, they enter a global enterprise landscape fraught with execution failures. To understand the true value of the architects emerging from this programme, we must look at the macroeconomic gap they are uniquely prepared to fill.

Enterprise technology investments globally often fail on the front lines when software is not properly operationalized, leading to millions in lost return on investment (ROI). To bridge this gap between high-level AI potential and operational reality, global technology leaders increasingly rely on Forward Deployed Engineers (FDEs). FDEs are elite, high-value professionals who combine software development, operational consulting, and frontline investigation by embedding directly into client workflows. Globally, the FDE role commands immense value because these practitioners solve execution failures that traditional back-office developers miss.

By some estimates, more than 80% of AI projects fail — twice the rate of IT projects that do not involve AI (RAND Corporation, 2024). Organizations routinely get trapped jumping on the hype bandwagon, launching projects that are poorly scoped and detached from operational realities. The companies in the successful minority understand that scaling is less about doing more and more about learning faster. They build tight feedback loops; they refine what works, quietly retire what doesn't, and tie experimentation directly to measurable outcomes.

To avoid this failure trap as it drives its "Big Push" Agenda for regional digital transformation, the Eastern Caribbean Central Bank (ECCB) is uniquely answering this need through the cutting-edge ECCB STEM Camp Project — specifically its Generative AI and Python Summer Camp Buildathon. This vision is explicitly championed by ECCB Governor Timothy N.J. Antoine, who has long argued that the region must be "producers, not mere consumers of knowledge and technology." Through this initiative, participants are empowered to move beyond simply using artificial intelligence to becoming creators, innovators, and problem-solvers prepared to lead the digital future of the region.

The Dual Engine — Product Engineers vs. Forward Deployed Engineers

Traditional technology education operates on a deeply flawed premise: it assumes that if you teach a student how to build a clean block of code in a sterile laboratory, they are ready to change the world.

They are not.

In the real world, code written in isolation shatters the moment it encounters the messy friction of human workflows, dirty data, and legacy corporate infrastructure. This friction is precisely why more than 80% of AI projects fail. To survive this gauntlet, a modern technology ecosystem cannot rely solely on back-office developers. It requires a tight, synchronized dance between two distinct identities: the Product Engineer and the Forward Deployed Engineer (FDE).

What made this year's ECCB STEM Camp Project a world-class anomaly is that it didn't choose between the two. It forced fourteen-to-eighteen-year-old Caribbean youth to become both.

Phase 1: The Product Engineers (Building the Paved Highway)

Inside the campus lab, the campers operated strictly as Product Engineers. Their mission was to build the core technical engine — the stable platform architecture. They had to design complex multi-agent systems, integrate diverse application programming interface (API) pipelines, and ensure system uptime. As Product Engineers, their focus was horizontal. They were building a scalable "paved highway" of generalized AI capabilities. They wrestled with syntax, mapped out memory allocation, and engineered foundational tools. But a paved highway is useless if it doesn't lead to a real destination.

Phase 2: The Forward Deployed Engineers (Paving the Gravel Road)

The true magic occurred when these same teenagers stepped out of the lab and onto the Buildathon stage to face real regional enterprises and public institutions. In that exact pivot, they became Forward Deployed Engineers. An FDE's focus is fiercely vertical. They do not care about theoretical code perfection; they care about operational execution. Straddling the line between software developer and frontline diplomat, they embedded themselves into the custom, unvarnished realities of local client workflows.

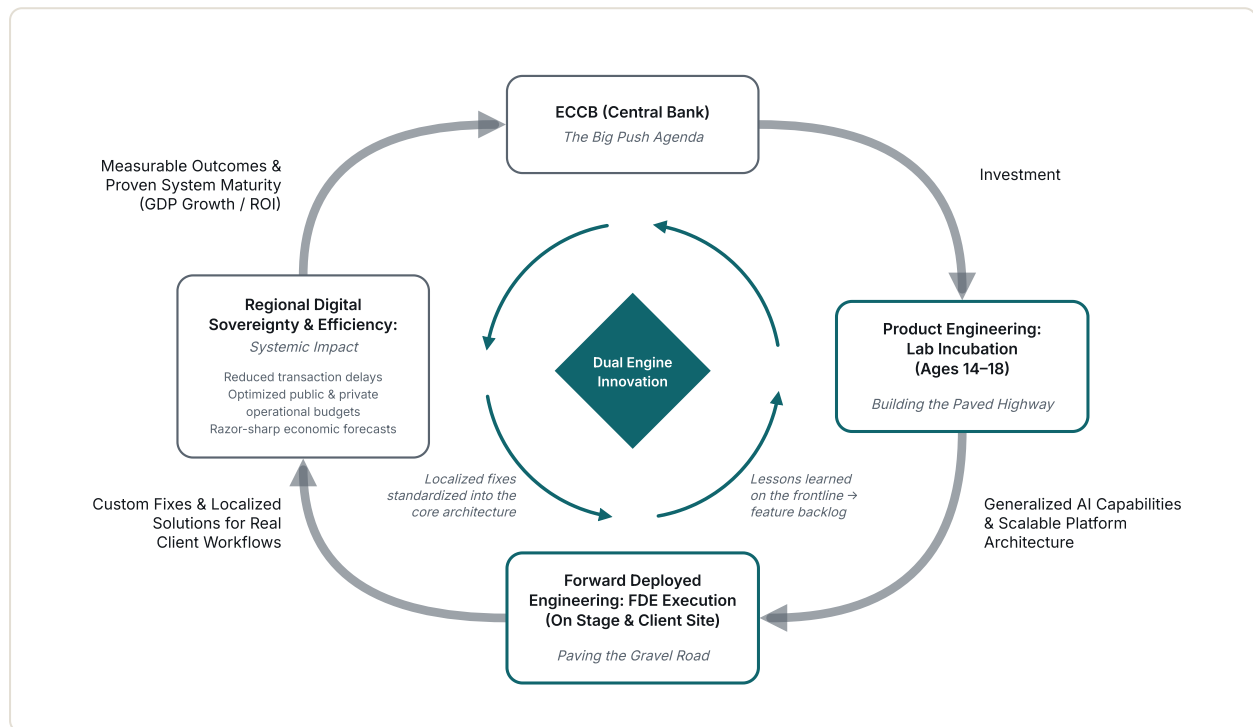
When the chief judge cross-examined a team on agents that had tangled in testing, the lab environment evaporated. The campers were on a rough "gravel road." They had to use rapid engineering intuition — crafting rigid system prompts and locking down output-

schema checkers — to map the advanced technology onto the client's immediate, high-stakes problem.

The Self-Sustaining Feedback Loop

By holding both identities, these youth did something most global tech enterprises struggle to achieve: they built a live engineering feedback loop.

FORGING AMBIDEXTROUS BUILDERS: THE DUAL ENGINE INNOVATION LOOP



The lessons learned on the frontline of client deployment traveled straight back into the lab. The bugs discovered during the high-pressure client showcase became the feature backlog for the next product build. The custom, localized fixes engineered on the fly were standardized and hardcoded into the platform's core architecture.

This is the dual engine that will drive the Eastern Caribbean's digital sovereignty. We are not just training youth to write code that sits on a shelf. We are forging a generation of ambidextrous builders: elite youth leaders who can architect a world-class product in the morning, and confidently deploy it into a complex, legacy enterprise by the afternoon.

Moving Beyond the "Junior Consultant"

Rather than training youth to be passive junior consultants who handle documentation and work in isolation from code, the camp prepares them as hands-on builders and problem-solvers. In traditional enterprise client engagement, junior roles act as mere reactive intermediaries. The youth emerging from this programme flip that dynamic completely. They are trained as Trusted Advisors capable of delivering evidence-based engagement that saves companies time, optimizes money, and ensures absolute execution.

What makes this paradigm shift staggering is that these FDE-minded builders are teenagers between 14 and 18 years old. At an age where traditional educational tracks expect passive listening, these youth step into roles typically reserved for seasoned analysts.

Predictive Enterprise Engagement

Scaling AI in the enterprise requires more than infrastructure; it requires rhythm. When FDE-minded builders embed an intentional workflow into local institutions, the enterprise client engagement strategy transforms completely. Client success turns proactive, anticipating operational issues and predicting bottlenecks before they happen.

The client engagement throughout the programme was a masterclass in best practice and continuous learning. It moved away from transactional exchanges, establishing a bidirectional learning loop between the young builders and real-world regional enterprises. Sybil Welsh, Senior Project Specialist at the ECCB and Project Manager for the camp, notes on this evolution:

"We are turning the tables on what it means to put on a camp. When you have a real client, what you do is immerse our young people in real-life skills, a real innovation lab. You have real clients, you have real projects, and now you really inspire our young people to do all that they need to do, to dig deep, to go beyond what they thought was possible."

This level of maturity shifts the entire open innovation process to an evidence-based engagement framework. It acts as a catalyst to shift public and private organizations from a reactive state to a predictive one. It builds deep, unshakeable trust — in the system, in the insights, and in the people using them. The process revealed that data collection,

model improvement, and user feedback must work together in a synchronized cycle. The result? Structural maturity. It's less about rapid expansion and more about building sustainable institutional foundations.

The Deep Plunge into Advanced Agentic Engineering

What was unique about this year's iteration is the deep plunge into the world of advanced agentic engineering. This is not about writing basic script prompts or riding a hype bandwagon; it is a rigorous, end-to-end Agentic Process that mirrors the highest tiers of global tech deployment:

- **Intent:** Accurately defining the enterprise problem and anchoring the system's objective.
- **Planning and Tooling:** Designing the architectural blueprint and provisioning the specific technical tools required for the task.
- **Execution with Tools:** Breaking down complex business workflows into automated, granular steps executed by specialized AI agents.
- **Reflection:** Evaluating system performance against the original goal, forcing the agents to self-correct until they meet the baseline intent.

When these multi-agent systems inevitably tangle or experience a loop during testing, these teenage programmers do not freeze. They use the failure as raw data to engineer rigid prompt guardrails and schema checkers. They understand that the only way to success is to understand the way through the problem.

Aspirational Outcomes: Scaling Growth through AI

The primary lesson learned from this buildathon process is that AI scaling is not a sprint; it is an evolution of regional trust. To truly transform the ecosystem and achieve the exponential growth targeted by the Big Push, public and private organizations must fundamentally change how they absorb technology. Growth will not scale by simply buying more software; it will scale by adopting the iterative, fearless learning loop demonstrated by our camp participants.

Right now, AI is a sensational headline topic. The ultimate aspiration is a shift in the regional mindset to where these FDE practices become standard, and AI becomes part of the invisible fabric of how the Eastern Caribbean works — quietly maximizing efficiency,

eliminating systemic bottlenecks, and optimizing resource allocation in the background, led by truly engaged, empowered, and competent youth leaders steering the region into the future.

In this mature ecosystem, regional executives will no longer talk about the AI hype bandwagon. Instead, they will speak directly about the real-world metrics that matter: reduced transaction delays, optimized public and private operational budgets, and razor-sharp economic forecasts. The conversation will permanently shift from the novelty of the tool to the power of the result. By anchoring this change in the technical grit and frontline leadership of our youth, the ECCB STEM Camp Project provides the blueprint to scale regional growth, turning the promise of the Big Push into an unvarnished, sovereign reality.

Sources

RAND Corporation (2024). [The Root Causes of Failure for Artificial Intelligence Projects and How They Can Succeed](#). "By some estimates, more than 80 percent of AI projects fail — twice the rate of failure for information technology projects that do not involve AI."

Eastern Caribbean Central Bank (ECCB). [Building a Digital Economy in the ECCU](#). Governor Timothy N.J. Antoine on the region as "producers, not mere consumers of knowledge and technology."

Eastern Caribbean Central Bank (ECCB), 1 September 2026. [ECCU Youth Harness AI Skills to Develop Solutions for Real-World Challenges](#). Sybil Welsh: "We are turning the tables on what it means to put on a camp."